```
import numpy as np
import tensorflow as tf

# Step 1: Load Pre-Trained YOLO Model
yolo_net = cv2.dnn.readNet('yolov4.weights', 'yolov4.cfg') layers = yolo_net.getLayerNames()
out_layers = [layers[i - 1] for i in yolo_net.getUnconnectedOutLayers()]

# Step 2: Load COCO Classes
with open('coco.names', 'r') as f:
    classes = f.read().strip().split('\n') # Step 3: Process Image
def detect_objects(image_path):
    image = cv2.imread(image_path)
    height, width = image.shape[:2]

    # Preprocess Input
    blob = cv2.dnn.blobFromImage(image, 1/255.0, (416, 416), swapRB=True, crop=False)
yolo_net.setInput(blob)

    # Run Detection
    outputs = yolo_net.forward(out_layers) boxes, confidences, class_ids = [], [], []

    for output in outputs:
        for detection in output:
            scores = detection[5:]
            class_id = np.argmax(scores)
            confidence = scores[class_id]

            if confidence > 0.5:
                box = detection[:4] * np.array([width, height, width, height]) (center_x, center_y,
box_w, box_h) = box.astype('int') x = int(center_x - (box_w / 2))
                y = int(center_y - (box_h / 2))
                boxes.append([x, y, int(box_w), int(box_h)]) confidences.append(float(confidence))
class_ids.append(class_id)

    # Apply Non-Max Suppression
    indices = cv2.dnn.NMSBoxes(boxes, confidences, 0.5, 0.4) for i in indices.flatten():
        x, y, w, h = boxes[i]
        label = str(classes[class_ids[i]])
        confidence = confidences[i]
        cv2.rectangle(image, (x, y), (x + w, y + h), (0, 255, 0), 2) cv2.putText(image, f"{label}:
{confidence:.2f}", (x, y - 10), cv2.FONT_HERSHEY_SIMPLEX, 0.5, (0, 255, 0), 2)
cv2.imshow('Detection', image)
    cv2.waitKey(0)
    cv2.destroyAllWindows()

# Example Usage
detect_objects('example_image.jpg')
```